

Coding Standards Document

COS 301 - Capstone 2026
Capstone Project: EquityLens
By The Big Five



Project Name: EquityLens
Client: Amazon Web Services (AWS)
Team: The Big Five (TB5)

Team members:

- Abdulrahman Sabah (Team Lead) - u24566170
- Joshua Heath - u23541475
- Antony van Straten - u24590739
- Jonty Honey - u23536862

Contents

1. Introduction
2. General Principles
3. Repository Structure
4. Version Control and Collaboration
5. Frontend Standards (JavaScript / React)
6. Backend Standards (Python / FastAPI)
7. Testing Standards
8. Security Standards
9. Documentation Standards
10. Tooling and Configuration Files

11. Conclusion

1. Introduction

This document describes the coding conventions and styles used across the EquityLens repository. It's purpose is to ensure, clarity, flexibility, reliability, and efficiency.

EquityLens is a mono-repository containing a React (JavaScript) frontend, a FastAPI (Python 3.11) backend and its supporting infrastructure and documentation.

2. General Principles

- Clarity. Code is written to be read by the next team member. Small, focused functions with intentional names.
- Consistency. When a pattern exists in the codebase the new code follows it.
- Explainable. Each area of the system has an owner and any line of code we merge must be explainable by its author in review and in the demo.
- Fail fast and degrade gracefully. Inputs are validated at the boundary (Pydantic schemas on the backend, form validation on the frontend). At runtime, the failure of one external dependency degrades only its own feature.
- Direct user-facing text. Error messages state what happened plainly.

3. Repository Structure

EquityLens is a single mono-repository, as required for the module.

Placement rules that keep this structure stable:

- A new API resource means a new router module, and its logic goes in a service, not in the router.
- A new page means a new directory under pages/. A reusable piece of UI means a directory under components/ containing the component and its test.
- Anything used by more than one feature lives in utils or common.

4. Version Control and Collaboration

4.1 Branching Strategy

- main always reflects a working, deployable version of the system.
- dev is the integration branch.
- Working branches are named [type]/[description]/[INITIALS], where type is one of feature, fix, docs, test, refactor, or chore, and INITIALS identifies the owner (AS, JH, JKH, AVS). Examples from the repository's history:

feature/registration/JH

feature/pipelines/AS

docs/srs-functional-requirements/JH

- Nobody commits directly to main or dev and all changes arrive by pull request.

4.2 Pull Requests and Review

- PR titles follow INITIALS : type: description (e.g. JH : feature: portfolio health score), so the project board and history read uniformly.
- Every PR requires two approving reviews before merge.
- The full CI pipeline must pass before a PR is eligible to merge.

5. Frontend Standards (JavaScript / React)

5.1 Language and Component Conventions

The frontend is plain JavaScript with JSX (no TypeScript). Type documentation is provided where it matters most:

- Components are function components only; class components are not used (the single exception is the `ErrorBoundary`, which React requires to be a class).
- One component per file; the file, the directory, and the component share the same PascalCase name with the component's test beside it.
- Presentational logic stays in the component and data fetching and multi-step flows live in hooks or services, so components remain testable with mocked data.
- Derived values that exist only to feed the are computed in pure helper modules under `utils/` and never inline in JSX.

5.2 Naming

- Components: PascalCase.
- Hooks: camelCase with a mandatory `use` prefix.
- Services and utilities: camelCase file names and exported functions are camelCase verbs.
- Variables and functions: camelCase, intention-revealing; single letters only for conventional loop indices.
- Constants: `UPPER_SNAKE_CASE` for module-level constants.
- Context providers: PascalCase ending in `Context`.

5.3 Styling and Theming

- Styling is TailwindCSS utility classes plus CSS custom properties. All colour, spacing and typography decisions route through the design tokens defined in `src/styles/globals.css`.
- Hard-coded colours are forbidden in components. Every colour reference is `var(--token-name)`. Before using a token, its exact name is checked against `globals.css`.
- The style guide page (`BrandStyleGuide.jsx`) documents the tokens but `globals.css` is authoritative when the two disagree.
- Animations respect the user's motion preference: `globals.css` contains a `prefers-reduced-motion`
- Interactive elements must be keyboard-reachable with visible focus states; semantic elements are used over clickable divs.

5.4 Comments and Error Handling

- Comments are sparse, lowercase, and explain why. A block comment is justified only for genuinely non-obvious decisions, such as a component's data contract or an algorithm's reasoning.

- API calls are wrapped in try/catch inside hooks/services and components receive { data, loading, error } shapes and render explicit loading, error and empty states. An error state always offers the user insight.
- User-facing error strings are direct and specific.

5.5 Linting

ESLint is configured in frontend/.eslintrc.cjs: eslint:recommended plus plugin:react/recommended.

6. Backend Standards (Python / FastAPI)

6.1 Layering Rules

The backend follows a strict three-layer structure and the layers are a coding standard.

- Routers (app/routers/) contain HTTP concerns only: path definitions, status codes, dependency injection, translating exceptions to HTTPException. A router never contains business logic and never touches the database directly.
- Services (app/services/) contain all business logic and orchestration. A service never imports from a router.
- Repositories (app/repositories/) own all queries for their aggregate. SQLAlchemy query logic outside a repository is a review defect.

6.2 Naming and Style

- Modules and functions: snake_case.
- Classes: PascalCase and SQLAlchemy models use PascalCase with snake_case __tablename__ and column names.
- Constants: UPPER_SNAKE_CASE at module level.
- One indicator per module under app/indicators/ each exposing a single calculation function.

6.3 Validation, Errors and Comments

- Every request and response body is a Pydantic schema in app/schemas/. Routers never accept raw dicts. Validation failures return 422 automatically.
- Services raise meaningful exceptions.
- External calls are isolated per request: a failure is caught in the owning service and degrades that feature only.
- Comment style matches the frontend: sparse and lowercase while explaining the non-obvious.

6.4 Database and Migrations

- Schema changes are made exclusively through Alembic migrations.
- Every migration must be reversible: upgrade() and downgrade() are both implemented and CI proves it on every run.

6.5 Linting and Static Analysis

ruff check . is the standard Python linter, run locally before every push (its cache is part of every developer's working tree). CI adds Bandit (security lint over app/), pip-audit (dependency vulnerabilities), and Trivy (repository-wide vulnerability, secret and misconfiguration scan) on every push and pull request.

7. Testing Standards

- Frontend unit/component tests use Vitest + React Testing Library, live beside the component they test and run with `npm run test`.
- Frontend end-to-end tests use Playwright from `frontend/e2e/`, run with `npm run test:e2e`
- Backend tests use pytest discovered from `backend/tests/` split into `unit/` and `integration/`. Coverage runs by default via `pytest.ini`'s `addopts`.
- Tests are named for behaviour, not implementation, so a failing test names the broken promise.

8. Security Standards

- No secrets in the repository. All .env files are git-ignored
- Backend code reads configuration only from the typed Settings class in app/config.py; os.environ access anywhere else is a review defect.
- Authentication is never hand-rolled. Identity, email verification and TOTP MFA are delegated to Cognito
- Every query for user-owned data filters by the authenticated user's id at the repository layer, otherwise treated as a critical defect.
- Portfolio data enters only by file upload or manual entry so we never ask for personal credentials
- Statement files are parsed client-side so raw statements are not transmitted.
- Dependency vulnerabilities surfaced by npm audit, pip-audit or Trivy.

9. Tooling and Configuration Files

The configurations that enforce these standards, all checked into the repository:

- frontend/.eslintrc.cjs - ESLint ruleset.
- frontend/vite.config.js - build configuration and Vitest setup: jsdom environment, global test APIs, shared setup file, lcov coverage output.
- frontend/playwright.config.js - E2E test directory and base URL for the local dev server.
- frontend/tailwind.config.js and frontend/postcss.config.js - utility-class and CSS build pipeline; design tokens themselves live in src/styles/globals.css.
- backend/pytest.ini - test discovery (testpaths = tests), asyncio mode, and default coverage flags.
- backend/requirements.txt / requirements-dev.txt - pinned runtime and development dependencies (pytest, coverage, Bandit, pip-audit).
- backend/alembic.ini + backend/alembic/ - migration environment.
- .github/workflows/ci.yml - the pipeline that runs lint, test, migration-reversibility and security-scan gates on every push and PR (diagrammed in SAS Section 5.3).
- docker-compose.yml + per-service.

11. Conclusion

These standards are the written form of how EquityLens is actually built resulting in a layered backend where query is user-scoped and a component frontend where every colour is a token and every prop boundary is documented, a repository where nothing reaches main without two reviews and a green pipeline. They are enforced by configuration and review rather than by memory.